

# Audio Module Documentation: Equalizer Block

A simple breakdown of how the 3-band equalizer splits our live microphone stream.

---

## 1. High-Level Diagram

This diagram tracks the parallel data routing inside the Equalizer Block. The single incoming audio block from the live microphone is copied simultaneously into three separate biquad filters (Low Pass, Band Pass, and High Pass) to break the spectrum down into Bass, Mids, and Treble frequencies.



## 2. Module Structures & Memory Footprint

---

The equalizer groups independent nested filter configurations to build a unified multi-band module architecture cleanly.

### Configuration Structure ( `equalizer_config_t` )

This aggregates individual coefficient datasets required by our underlying biquad cascades during startup configuration steps.

- `low_pass_config_t low` : Holds the 5 standard filter coefficient values (`b_0`, `b_1`, `b_2`, `a_1`, `a_2`) for the Bass processing block.
- `band_pass_config_t mid` : Holds the 5 standard coefficient values targeting the Mid range block.
- `high_pass_config_t high` : Holds the 5 standard coefficient values mapping out the Treble frequency block.

### Workspace Handle Structure ( `equalizer_hdl_t` )

Houses persistent instance states. It maintains structural memory isolations for each active sub-module channel concurrently.

- `low_pass_hdl_t low_hdl` : State variables and history buffers mapping the Bass cascade pipeline.
- `band_pass_hdl_t mid_hdl` : State variables tracking the voice and speech frequency range.
- `high_pass_hdl_t high_hdl` : State tracking details for high crisp brilliance or air bands.

### Static Footprint Analysis

| Sub-Module Target Tracker  | Internal Data Arrays            | Byte Weight Footprint | Dynamic Heap Overhead        |
|----------------------------|---------------------------------|-----------------------|------------------------------|
| low_hdl (Bass Sub-unit)    | State history mapping variables | Variable (~40 bytes)  | 0 Bytes (Statically Managed) |
| mid_hdl (Mid Sub-unit)     | State history mapping variables | Variable (~40 bytes)  | 0 Bytes (Statically Managed) |
| high_hdl (Treble Sub-unit) | State history mapping variables | Variable (~40 bytes)  | 0 Bytes (Statically Managed) |

### 3. Simple Parallel Math Logic

Unlike serial modules, a standard equalizer block uses a parallel topology. It does not overwrite a single stream. Instead, it reads the input stream and feeds it into three mathematically independent channels simultaneously.

The processing architecture loops through frames implementing these three tasks in parallel:

```

Bass_Output[n] = Filter_Low(Input[n])
Mids_Output[n] = Filter_Mid(Input[n])
Treble_Output[n] = Filter_High(Input[n])

```

By outputting three isolated frequency streams instead of immediately summing them together inside the loop, the calling routine gains total freedom to change the volume of each band individually using external gain blocks before final mixing occurs.

## 4. Lifecycle Functions & Loop Execution

---

The equalizer execution layout follows our standard lifecycle plan. We treat pointer overlaps carefully. Because this module writes out three distinct destination arrays simultaneously, passing shared tracking arrays can lead to memory clobbering. This behavior contract is explicitly detailed below and documented directly in the public header.

### A. Query Memory Size ( `equalizer_open` )

```
STATUS equalizer_open(uint32_t *pui32Size) {
    if (pui32Size == NULL) return STATUS_NOT_OK;
    *pui32Size = sizeof(equalizer_hdl_t);
    return STATUS_OK;
}
```

**What it does:** Tells our framework exactly how many bytes are needed to allocate the structural handle instance safely.

### B. Initialize Module Settings ( `equalizer_init` )

```
STATUS equalizer_init(equalizer_hdl_t *phdl, const equalizer_config_t *psConfig) {
    if (phdl == NULL || psConfig == NULL) return STATUS_NOT_OK;

    low_pass_init(&phdl->low_hdl, &psConfig->low);
    band_pass_init(&phdl->mid_hdl, &psConfig->mid);
    high_pass_init(&phdl->high_hdl, &psConfig->high);

    return STATUS_OK;
}
```

**What it does:** Links the nested sub-handles and passes the starting frequency coefficient groups down into their respective initialization targets.

### C. Process Audio Samples ( `equalizer_process` )

```
/**
 * @brief Splits an input stream into three isolated frequency bands.
```

```

* @note [NON-IN-PLACE COMPATIBLE] This function writes out multiple streams
* simultaneously in parallel. The output target buffers (pfLowOutput, pfMidOutput,
* and pfHighOutput) must point to independent regions and cannot share pointers
* with pfInput or each other.
*/
STATUS equalizer_process(equalizer_hdl_t *phdl,
                        const float *pfInput,
                        float *pfLowOutput,
                        float *pfMidOutput,
                        float *pfHighOutput,
                        uint32_t ui32NumSamples) {
    if (phdl == NULL || pfInput == NULL || pfLowOutput == NULL || pfMidOutput == NULL ||
pfHighOutput == NULL) {
        return STATUS_NOT_OK;
    }

    low_pass_process(&phdl->low_hdl, pfInput, pfLowOutput, ui32NumSamples);
    band_pass_process(&phdl->mid_hdl, pfInput, pfMidOutput, ui32NumSamples);
    high_pass_process(&phdl->high_hdl, pfInput, pfHighOutput, ui32NumSamples);

    return STATUS_OK;
}

```

**What it does:** Feeds the current live microphone audio frame into each filter path. Safety depends strictly on implementation loops. Since this function splits one stream into three separate arrays, it is 'strictly non-in-place'. The output arrays must be completely isolated to avoid corrupting data mid-loop.

## D. Close and Clean Up ( `equalizer_close` )

```

STATUS equalizer_close(equalizer_hdl_t *phdl) {
    if (phdl == NULL) return STATUS_NOT_OK;

    low_pass_close(&phdl->low_hdl);
    band_pass_close(&phdl->mid_hdl);
    high_pass_close(&phdl->high_hdl);

    return STATUS_OK;
}

```

**What it does:** Safely closes out the nested sub-units. Since all workspace state handles are statically allocated across the scope, no dynamic `free()` calls are executed.